

React — 复习要点

1 React 简介 (Introduction to React)

1.1 React 的由来与背景 (Origin & Background)

- 2011 年，Facebook 工程师 **Jordan Walke** 创建 React，应对新闻动态页面日益复杂的挑战；2013 年正式开源。 [p.4]
- 当时面临的核心问题：
 - 代码复杂度急剧上升：大量 DOM 操作与业务逻辑混杂在一起，难以维护。 [p.4]
 - 数据同步困难：多个组件共享状态时，难以保持数据一致性。 [p.4]
 - 性能问题：频繁的 DOM 更新导致页面卡顿。 [p.4]

1.2 React 的设计理念 (Design Philosophy)

- **Learn Once, Write Anywhere**（学习一次，随处编写）：同一套思想可用于 Web、移动端 (React Native)、桌面应用 (Electron + React) 等多种平台。 [p.5]
- **组件化思想** (Component-Based)：将复杂的 UI 拆分为独立的、可复用的组件。 [p.5]
- **状态驱动视图** (State-Driven UI)：UI 是状态的函数，状态变化自动更新视图。 [p.5]
- **核心定位**：专注于视图层 (View Layer)，提供声明式 (Declarative) 方式描述 UI，可与任意后端技术栈配合。 [p.5]

1.3 React 的主要功能特性 (Key Features)

- **声明式编程** (Declarative Programming)：描述 UI 在不同状态下的样子，React 自动处理 DOM 更新，避免手动 DOM 操作，代码更易理解。 [p.6]
- **组件化开发** (Component-Based Development)：将 UI 拆分为独立、可复用的组件，提高代码复用性，便于团队协作。 [p.6]
- **虚拟 DOM** (Virtual DOM)：高效的 DOM 更新策略，减少直接 DOM 操作，提升性能，避免频繁重排 (Reflow) 和重绘 (Repaint)。 [p.6]
- **单向数据流** (Unidirectional Data Flow)：数据流向清晰可预测，易于调试和维护，避免数据混乱。 [p.6]
- **JSX 语法**：在 JavaScript 中编写类 HTML 结构，提升开发效率，增强代码可读性。 [p.6]

1.4 库与框架的区别 (Library vs. Framework)

- React 是**库 (Library)**，非**框架 (Framework)**：只负责 UI 的渲染和更新，不强制项目结构。 [p.7]
- **控制权对比**：
 - 库：开发者调用库的 API；低约束，灵活使用；关注解决特定问题；易于替换和组合。 [p.7]

- 框架：框架控制执行流程；高约束，遵循约定；提供完整解决方案；难以替换核心模块。
[p.7]
- React 无侵入性：可逐步引入现有项目，可与其他框架共存，不强制改变现有代码结构。 [p.7]
- 灵活技术选型：路由 (React Router、Next.js)、状态管理 (Context、Redux、Zustand、Jotai)、HTTP 请求 (axios、fetch、SWR、React Query) 均可自由选择。 [p.7]
- 对比 Angular（框架）：强制使用 TypeScript、强制特定项目结构、内置路由/状态管理/HTTP 客户端、遵循严格设计模式和约定。 [p.8]

2 JSX (JavaScript XML)

2.1 为什么需要 JSX? (Why JSX?)

- React 之前，前端主要采用两种方式构建 UI：手动 DOM 操作（命令式）或模板字符串拼接，可读性差、效率低下。 [p.10]
- JSX 将 HTML 结构直接嵌入 JavaScript，兼顾可读性和开发效率，提供了一种更优雅的 UI 描述方式。 [p.10]

2.2 JSX 的定义与本质 (Definition & Essence)

- **JSX (JavaScript XML)**：JavaScript 的扩展语法，允许在 JavaScript 代码中编写类似 HTML 的结构。 [p.11]
- 本质：JSX 不是浏览器原生支持的语法，会被 **Babel 编译器**转换为普通的 JavaScript 函数调用 (`React.createElement`)。 [p.11]
- 编程范式对比：
 - 命令式编程 (Imperative)：告诉计算机“怎么做”，详细描述执行步骤。 [p.11]
 - 声明式编程 (Declarative)：告诉计算机“做什么”，描述目标状态。 [p.11]

2.3 JSX 核心语法规则 (Core Syntax Rules)

- 必须有根元素包裹：一个 JSX 表达式只能返回一个根元素，这是 React 的设计约束，便于统一处理。 [p.12]
- 标签必须闭合：自闭合标签使用 `</>`，避免 HTML 语法错误。 [p.12]
- 使用 `className` 替代 `class`：`class` 是 JavaScript 关键字，避免语法冲突。 [p.12]
- 使用 `htmlFor` 替代 `for`：`for` 是 JavaScript 关键字，避免语法冲突。 [p.12]
- 大括号 `{}` 内写表达式：`{}` 内可以写任意 JavaScript 表达式，实现动态内容渲染。 [p.12]

2.4 JSX 的编译过程 (Compilation Process)

- 编译流程：JSX 源代码 → Babel 编译 → `React.createElement(type, props, ...children)` 调用 → 返回虚拟 DOM 对象 (JavaScript Object)。 [p.14]
- 本质：JSX 是 `React.createElement` 的语法糖 (Syntactic Sugar)。 [p.14]

3 组件化开发 (Component-Based Development)

3.1 组件化的基本概念 (Core Concepts)

- 将复杂的 UI 界面拆分为独立的、可复用的组件，每个组件负责自己的逻辑和样式。 [p.16]
- 组件化的优势：

- **高复用性**：一次编写，多处使用。 [p.16]
- **低耦合**：组件之间相互独立，便于维护。 [p.16]
- **易测试**：组件可以独立进行单元测试。 [p.16]
- **可维护性**：代码结构清晰，易于定位问题。 [p.16]
- **团队协作**：不同开发者可以负责不同组件的开发。 [p.16]

3.2 函数组件 (Function Components)

- 函数组件是定义组件的主要方式：使用 JavaScript 函数返回 JSX。 [p.17]
- 组件命名规范：
 - 必须以大写字母开头，区分组件和普通 HTML 标签。 [p.17]
 - 使用 **PascalCase**（大驼峰命名法）。 [p.17]
 - 文件名与组件名一致，便于查找和维护。 [p.17]

3.3 Props —— 组件间数据传递 (Props: Data Transfer)

- **Props** (Properties 的缩写)：组件间传递数据的方式，从父组件流向子组件。 [p.18]
- Props 的核心特性：
 - **只读性** (Read-Only)：子组件不能修改 props，保证数据流向的单向性。 [p.19]
 - **单向传递** (Unidirectional)：只能从父组件传递到子组件，简化数据流追踪。 [p.19]
 - **类型检查** (Type Checking)：可使用 PropTypes 或 TypeScript，提高代码健壮性。 [p.19]
 - **默认值** (Default Values)：可设置默认属性值，增强代码健壮性。 [p.19]
- **子传父**：子组件通过回调函数 (Callback Function) 向父组件传递数据。 [p.20]

4 状态管理——useState (State Management)

4.1 为什么需要状态管理? (Why State Management?)

- 组件中存在动态变化的数据：用户输入的表单数据、购物车商品数量、按钮点击次数等。 [p.22]
- 使用普通变量存储这些数据，数据变化不会触发组件重新渲染，UI 无法自动更新。 [p.22]

4.2 useState Hook

- **State** 是一种特殊的数据存储机制，当 State 变化时 React 会自动重新渲染组件（响应式, Reactive）。 [p.23]
- State 的核心特性：
 - State 是**组件内部的可变数据**（组件私有，外部无法直接访问），用于存储组件的动态信息。 [p.23]
 - State **不可直接修改**，必须通过 `setState` 函数更新（Hook 机制）。 [p.23]
 - 一个组件中若有多个 State，React 会**批量更新** (Batched Update)，以提高性能（异步更新, Asynchronous Update）。 [p.23]
- **基本语法**：`const [state, setState] = useState(initialValue);` [p.23]
- **参数说明**：
 - **initialValue**：初始值，可以是任意类型（数字、字符串、对象、数组等）。 [p.23]
 - **state**：当前状态值。 [p.23]
 - **setState**：更新状态的 Hook 函数。 [p.23]

- **异步更新注意事项:**连续多次调用 `setState` 可能不会立即反映最新值,应使用函数式更新 `setState(prev => prev + 1)` 获取前一次状态。 [p.25]

5 副作用处理——useEffect (Side Effects)

5.1 什么是副作用? (What are Side Effects?)

- **useEffect** (副作用 Hook 函数): 处理与组件渲染无关的操作, 通过 Hook 实现。 [p.27]
- 常见副作用操作:
 - **数据获取 (Data Fetching):** 调用 API 获取数据。 [p.27]
 - **订阅事件 (Event Subscription):** WebSocket 订阅、事件监听。 [p.27]
 - **DOM 操作 (DOM Manipulation):** 修改 document title、操作 DOM 元素。 [p.27]
 - **定时器 (Timers):** `setTimeout`、`setInterval`。 [p.27]
 - **日志记录 (Logging):** 发送分析数据到服务器。 [p.27]

5.2 useEffect 依赖数组与执行时机 (Dependency Array & Execution Timing)

- **空数组 []:** 仅在组件挂载 (Mount) 时执行一次。适用场景: 初始化操作、一次性数据获取。 [p.28]
- **有依赖 [dep]:** 挂载时执行, 且每次依赖数据变化时重新执行。适用场景: 依赖特定数据的操作。 [p.28]
- **不写依赖数组:** 每次组件渲染 (Render) 时都执行。适用场景: 需要同步更新的操作。 [p.28]

5.3 useEffect 清理函数 (Cleanup Function)

- **useEffect** 中可以返回一个函数, 在组件卸载 (Unmount) 时或下次 effect 执行前调用。 [p.28-30]
- 用于清除定时器、取消订阅、清理事件监听等资源释放操作。 [p.28-30]

5.4 useState 与 useEffect 对比

- **useState:** 管理状态数据, 存储和更新组件内部的数据。 [p.31]
- **useEffect:** 处理副作用, 执行与渲染无关的操作, 管理副作用的执行时机。 [p.31]

6 虚拟 DOM (Virtual DOM)

6.1 虚拟 DOM 简介 (Introduction)

- **虚拟 DOM (Virtual DOM):** 真实 DOM 的 JavaScript 对象表示形式, 即内存中的另一个 DOM 树对象。 [p.33]
- **为什么需要虚拟 DOM:** 直接操作真实 DOM 是非常昂贵的操作, 每次 DOM 操作都会触发浏览器的重排 (Reflow) 和重绘 (Repaint), 这是性能瓶颈的主要来源。 [p.33]

6.2 虚拟 DOM 工作原理 (Working Principle)

- **首次渲染 (Initial Render):** React 根据 JSX 创建虚拟 DOM 树, 然后将其渲染为真实 DOM。 [p.33]
- **状态更新 (State Update):** 当组件状态变化时, React 创建一个新的虚拟 DOM 树。 [p.33]
- **Diff 算法 (Diff Algorithm):** React 使用高效的 Diff 算法对比新旧虚拟 DOM 树的差异。 [p.33]
- **批量更新 (Batched Update):** React 计算出最小的 DOM 更新操作, 然后批量应用到真实 DOM 上。 [p.33]

6.3 Diff 算法核心策略 (Core Diff Strategies)

- **同层比较 (Same-Level Comparison):** 只比较同一层级的节点，不跨层级比较，时间复杂度从 $O(n^3)$ 降至 $O(n)$ 。 [p.34]
- **类型不同则替换 (Type Mismatch $\rightarrow$ Replace):** 新旧节点类型不同时，直接替换整个子树，不再递归比较子节点。 [p.34]
- **key 属性优化 (Key Optimization):** 通过 key 属性唯一标识列表中的每个元素，使 Diff 算法能精确识别元素的增删和移动，避免不必要的重建。 [p.34]

7 组件生命周期 (Component Lifecycle)

7.1 类组件生命周期 (Class Component Lifecycle)

- **挂载阶段 (Mounting):** `constructor()` $\rightarrow$ `render()` $\rightarrow$ `componentDidMount()` [p.28]
- **更新阶段 (Updating):** `render()` $\rightarrow$ `componentDidUpdate(prevProps, prevState)` [p.28]
- **卸载阶段 (Unmounting):** `componentWillUnmount()` ——用于清理定时器、取消订阅等。 [p.28]

7.2 函数组件中的生命周期 (Lifecycle in Function Components)

- **`componentDidMount`** $\leftrightarrow$ `useEffect` 空依赖数组 `[]`: 仅在挂载时执行。 [p.28]
- **`componentDidUpdate`** $\leftrightarrow$ `useEffect` 有依赖 `[dep]`: 依赖变化时执行。 [p.28]
- **`componentWillUnmount`** $\leftrightarrow$ `useEffect` 返回的清理函数 (Cleanup Function): 组件卸载前执行。 [p.28]
- 函数组件推荐使用 Hooks 替代传统生命周期方法。 [p.27-28]

8 其他常用 Hooks (Other Common Hooks)

8.1 `useRef`

- 用于保存可变值 (Mutable Value)，其变化不会触发组件重新渲染。 [p.27]
- 常用于获取 DOM 元素引用、保存定时器 ID、存储上一次的值等。 [p.27]

8.2 `useMemo` 与 `useCallback`

- **`useMemo`:** 缓存计算结果 (Memoized Value)，仅在依赖变化时重新计算，避免每次渲染都执行昂贵计算。 [p.27]
- **`useCallback`:** 缓存函数引用 (Memoized Callback)，仅在依赖变化时重新创建函数，避免子组件不必要的重渲染（配合 `React.memo` 使用）。 [p.27]

8.3 `useContext`

- 在函数组件中消费 Context 值，替代 `Context.Consumer` 的 `render props` 写法。 [p.7,37]

9 React 最佳实践 (React Best Practices)

- **单一职责原则 (Single Responsibility Principle):** 每个组件只负责一个功能，保持组件简洁和可维护。 [p.35]
- **状态提升 (Lifting State Up):** 将共享状态提升到最近的公共祖先组件中管理，保持数据流的单向性。 [p.35]

- **代码组织规范 (Code Organization)**: 合理的文件结构, 组件、样式、测试文件就近放置。[p.35]
- **命名规范 (Naming Convention)**: 组件使用 PascalCase, 文件名与组件名一致; 普通函数/变量使用 camelCase。[p.17,35]
- **导入顺序 (Import Order)**: 第三方库优先 → 绝对路径导入 → 相对路径导入 → 样式文件放在最后。[p.35]
- **使用 key 属性**: 列表渲染时为每个元素设置唯一稳定的 key, 帮助 React 高效更新列表。[p.34]
- **避免直接修改 State**: 始终使用 setState / setXxx 函数更新状态, 保持不可变更新 (Immutable Update)。[p.23]

10 MPA 与 SPA 架构 (MPA vs. SPA Architecture)

10.1 概念定义 (Definitions)

- **MPA (Multi-Page Application, 多页面应用)**:
 - 传统的 Web 应用架构, 每个页面都是独立的 HTML 文件。[p.37]
 - 流程: 用户点击链接 → 浏览器发送请求 → 服务器返回新 HTML → 浏览器重新渲染整个页面。[p.37]
- **SPA (Single-Page Application, 单页面应用)**:
 - 现代前端应用架构, 只有一个 HTML 文件, 所有内容通过 JavaScript 动态更新。[p.37]
 - 流程: 用户首次访问 → 加载唯一的 HTML → 后续导航通过 JavaScript 处理 → 仅更新部分页面。[p.37]
- **SPA 的核心技术**:
 - **前端路由 (Frontend Routing)**: 如 React Router、Vue Router。[p.37]
 - **虚拟 DOM (Virtual DOM)**: 高效的局部更新。[p.37]
 - **状态管理 (State Management)**: 如 Redux、React Context。[p.37]

10.2 MPA 与 SPA 架构对比 (Comparison)

- **页面加载方式**: MPA 每次导航重新加载整个页面; SPA 首次加载后仅更新局部内容。[p.38]
- **用户体验**: MPA 有页面刷新感; SPA 流畅无刷新。[p.38]
- **首屏加载时间**: MPA 较快 (只需加载当前页面); SPA 较慢 (需加载完整应用)。[p.38]
- **SEO 友好性**: MPA 天然友好 (每个页面独立 URL); SPA 需要额外处理 (SSR / SSG, 服务端渲染 / 静态站点生成)。[p.38]
- **开发复杂度**: MPA 较低; SPA 较高 (需处理路由、状态等)。[p.38]
- **资源复用**: MPA 低 (重复加载公共资源); SPA 高 (一次加载, 多次复用)。[p.38]
- **适用场景**: MPA 适合简单网站、内容型网站; SPA 适合复杂应用、交互型应用。[p.38]

11 本章小结 (Chapter Summary)

- React 是一个用于构建用户界面的 JavaScript 库, 专注于视图层。[p.5,40]
- 核心知识体系包括: 声明式编程与 JSX、组件化开发 (函数组件 + Props)、状态管理 (useState)、副作用处理 (useEffect)、虚拟 DOM 与 Diff 算法。[p.40]
- React 作为库的特点: 灵活、无侵入、可逐步引入; 与 Angular 等框架的设计思路不同。[p.7-8]
- MPA 与 SPA 是两种主要的 Web 应用架构模式, 各有优劣和适用场景。[p.37-38]

- 通过丰富的生态系统（路由、状态管理、HTTP 请求等），React 可组合成完整的开发解决方案。
[p.7]